

April 20, 2026

Rochester Institute of
Technology
CSCI.355.02

Machine Learning Final Project Presentation

Quick to Draw? More Like Quick to Classify!

Group 4: Dylan Shaffer, Sam Moll, Michael Reizenstein



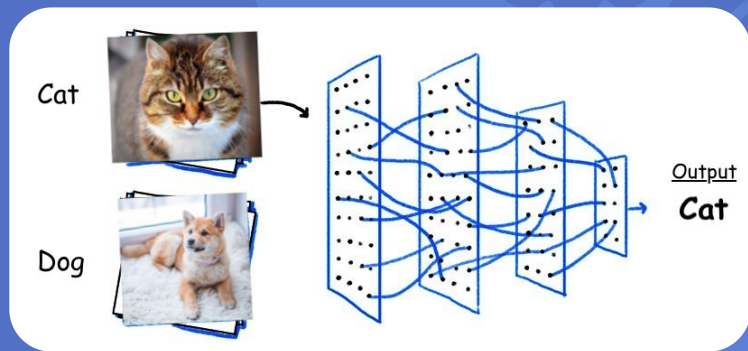
01

Introduction

Problem, Background,
Hypotheses

- **Image classification**

- **How well can different models recognize and distinguish between simple hand-drawn objects?**
 - Inconsistent, noisy, very abstract
 - Lot of variation in style and detail
 - Models must learn the general patterns through the noise



Guess the Classes!

Can you guess what these doodles represent?



Guess the Classes!

That's right! It's a ...



Cat

Guess the Classes!

That's right! It's a ...



Cat



Dog



Guess the Classes!

That's right! It's a ...



Cat



Dog



Bread

Problem Question

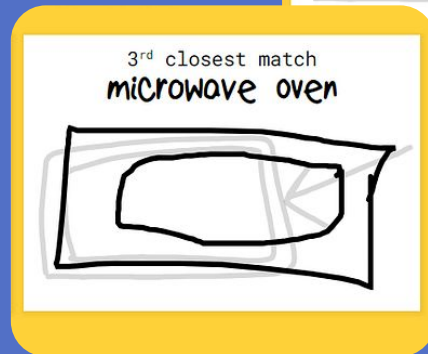
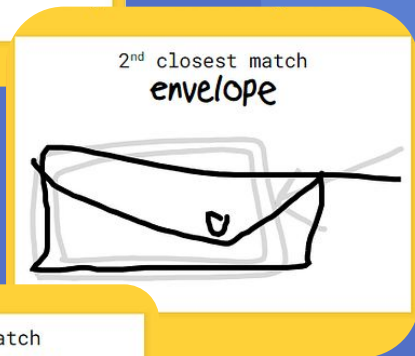
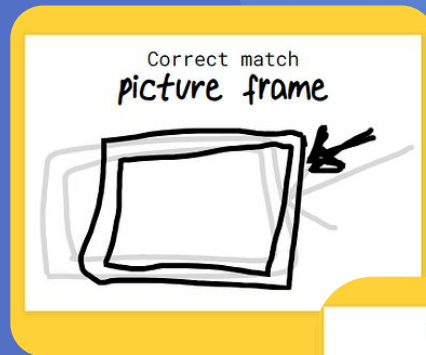


- **Using a dataset of hand-drawn images, will explore implementation of different models to compare and contrast performance**
 - How long does it take to train the model?
 - How accurately does the model classify the images?
 - Can the model distinguish between similarly drawn objects?

Hypothesis

- **Main Hypothesis** → **Model performance will vary depending on the visual similarity between classes**
 - Models will have higher accuracy on visually distinct classes
 - Models will struggle with similar classes with frequent misclassifications

Expect that the most challenging aspect of task will distinguishing between objects that humans draw in very similar ways!





02

Methods

Dataset, Preprocessing,
Design, Tools, Models,
Architecture, Evaluation

Chosen Dataset



Can a neural network learn to recognize doodling?

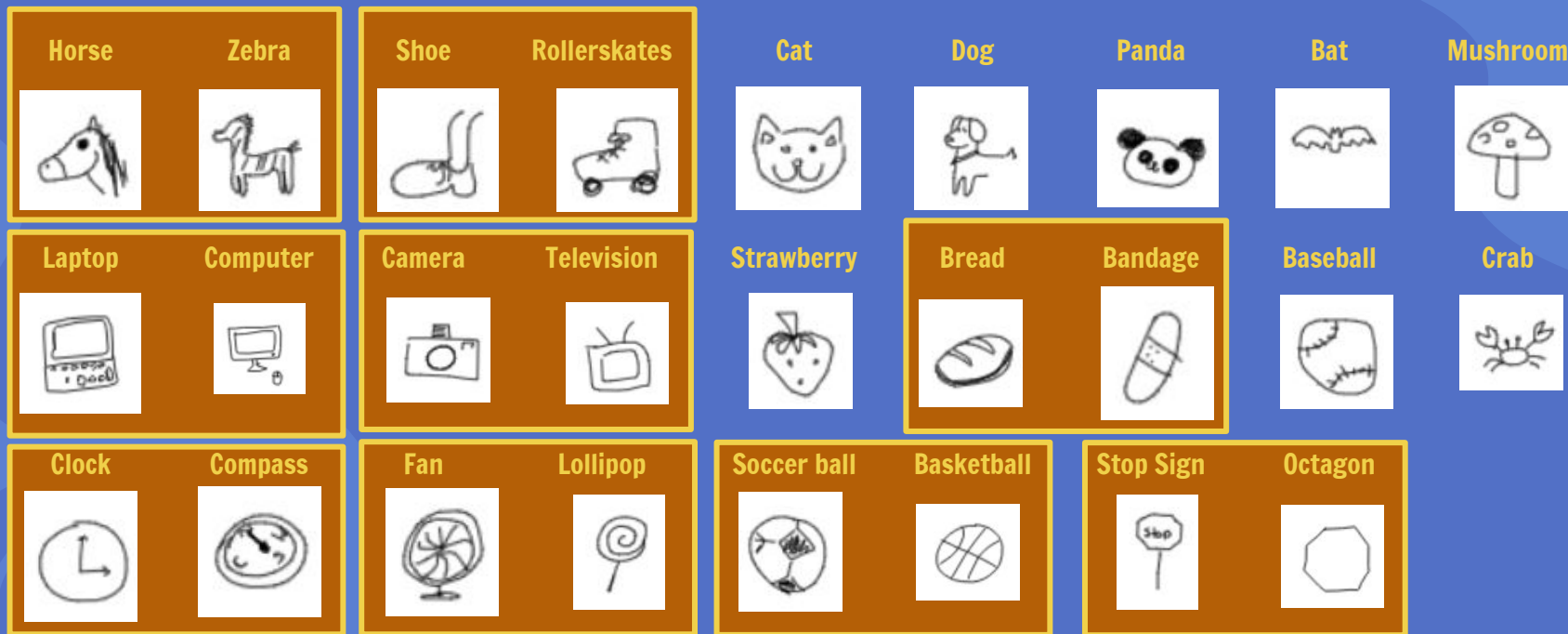
Help teach it by adding your drawings to the [world's largest doodling data set](#), shared publicly to help with machine learning research.

- **Google's Quick, Draw! Dataset**
 - Contains millions of hand-drawn sketches collected from users
 - Each drawing is stored as a sequence of vector strokes (lists of x and y coordinates that represent how the drawing was created)
- **Selected a subset of 26 classes**

We chose clusters of classes to try and see if similar classes would have higher cross-confusion:

- Horse and Zebra
- Computer and Laptop
- Baseball, Basketball, and Soccerball

Chosen Subset of Dataset



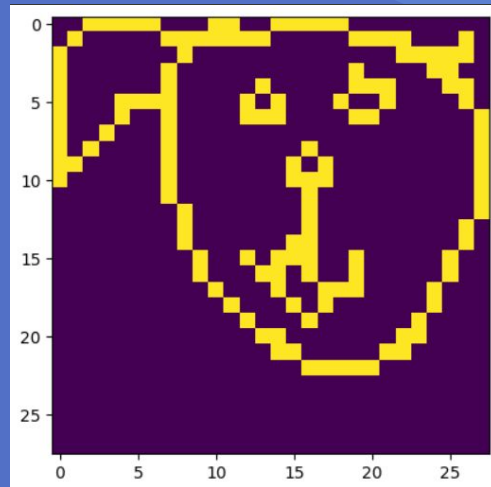
- Limit each epoch
- Train each of numbers to a
 - 100 ; 1,000

- Dataset originally stored as vector strokes

- pixels

- **Random undersampling to balance dataset**

- 100 ; 1,000 ; 10,000

[illegible]

Overview of Models

01

CNN

Convolutional
Neural
Network

02

SVM

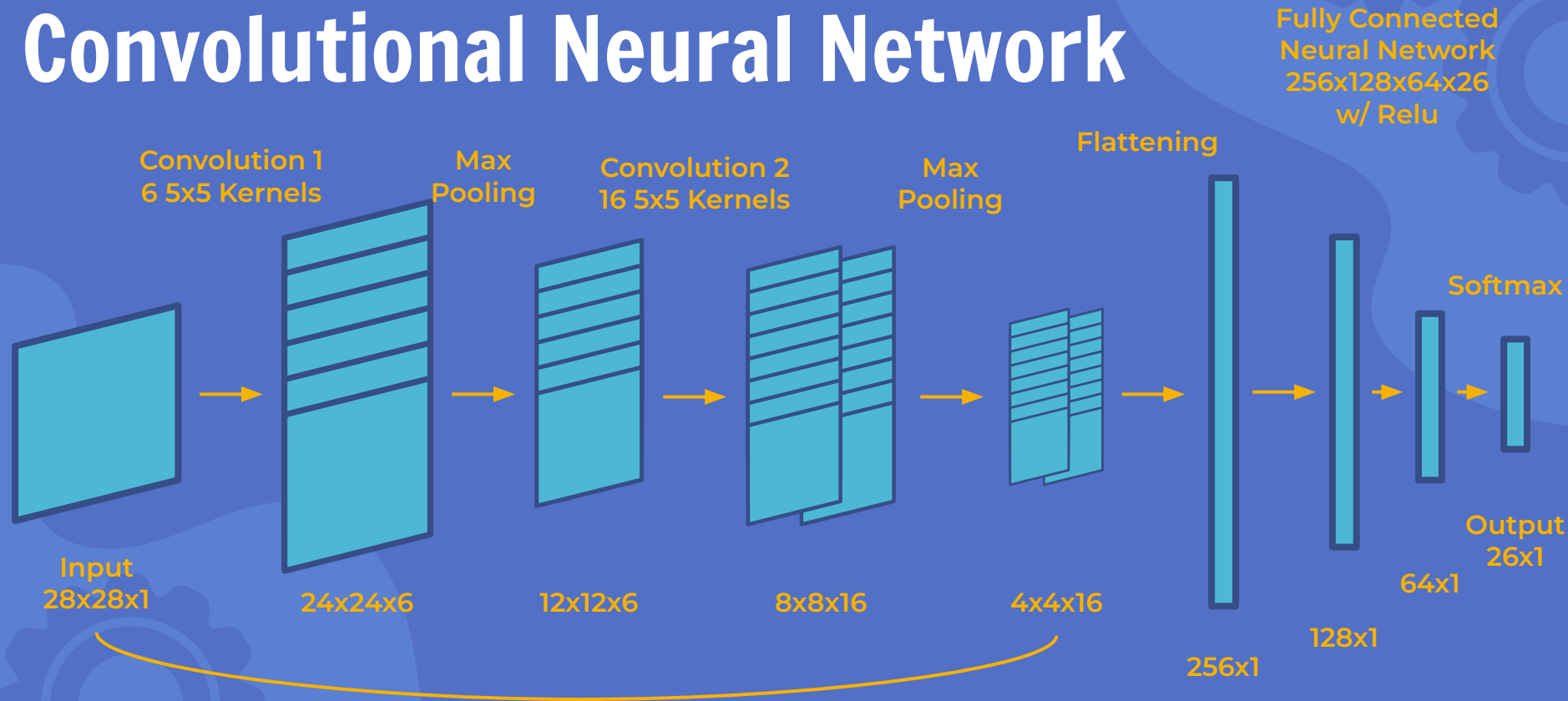
Support
Vector
Machine

03

ViT

Vision
Transformer

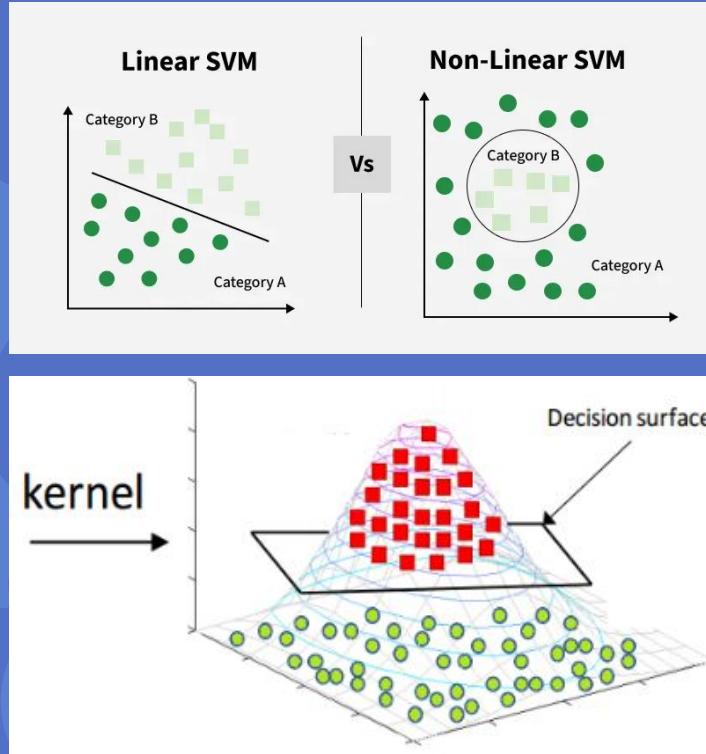
Convolutional Neural Network



Convolutional Neural Network (CNN)

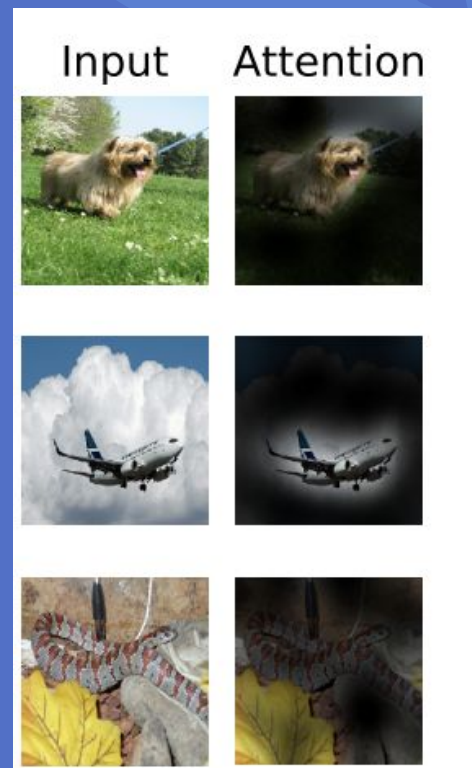
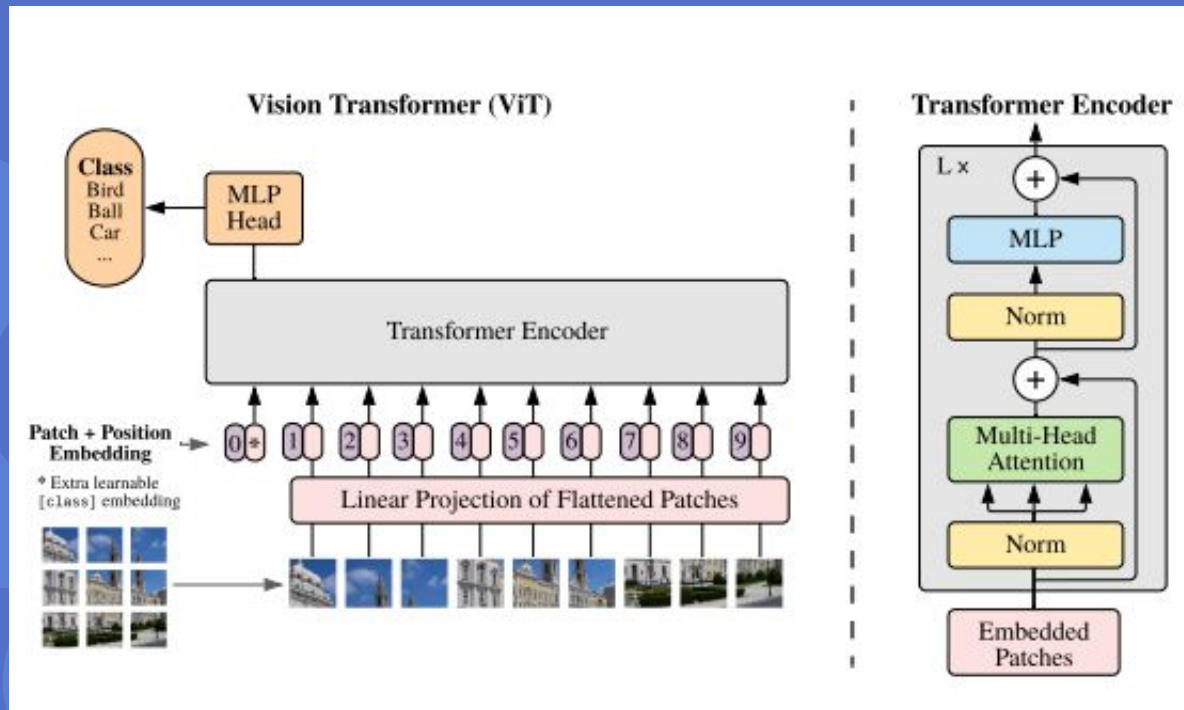
- Images have a size of 28x28
- Used the LeNet model
- As our data also had a 28x28 pixel input size, I used the LeNet CNN Model from the MNIST dataset as a baseline format for the dimensions of our convolutions.
-

SVM with RBF



- **Feature scaling using StandardScaler**
 - Mean = 0 and Variance = 1
- **Dimensionality reduction using Principal Component Analysis (PCA)**
 - Each image = 784 features (28×28)
 - Reduce to 100 features (randomized)
- **Trained model using SVM w/ Radial Basis Function (RBF) kernel**
 - Linear SVM had low accuracy
 - RBF allows non-linear decision boundaries between classes
- **Regularization parameter C = 1**
 - Margin maximization & classification error
 - C=10 had slight overfitting

Vision Transformer (ViT)



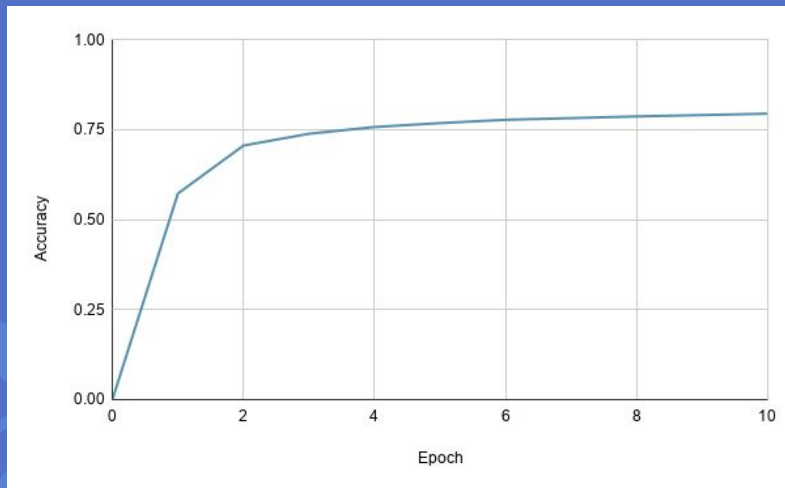


03

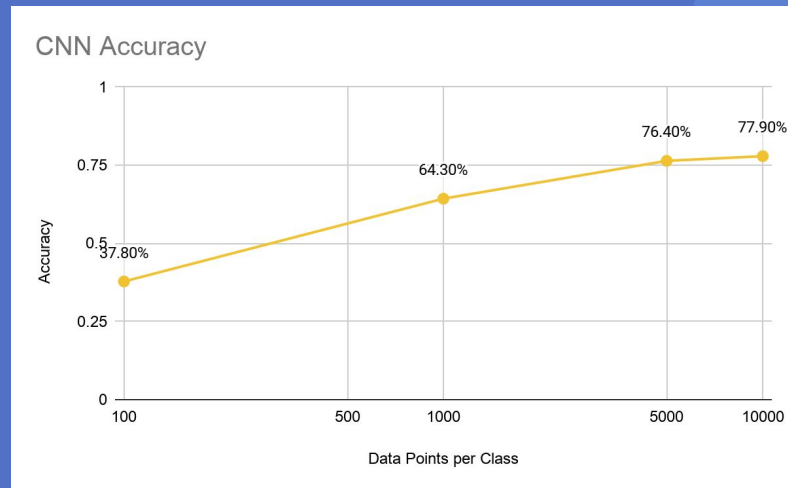
Results

CNN Training Results

Epoch vs. Accuracy

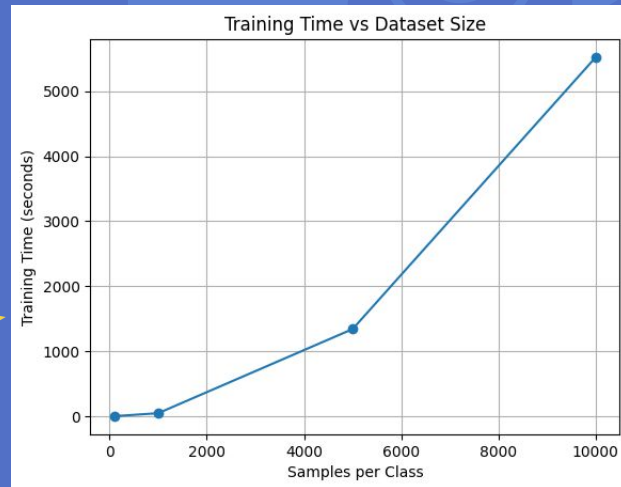


Datapoints vs. Accuracy

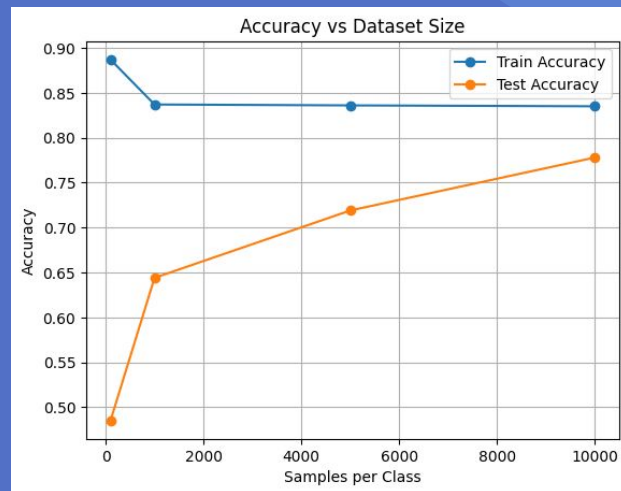


SVM Training Results

- **Training Times** (*Samples per 26 Classes)
 - 100 Samples; 1.65 seconds
 - 1,000 Samples; 47.13 seconds
 - 5,000 Samples; 1341.05 seconds (22 min)
 - 10,000 Samples; 5521.07 seconds (92 min)



- **Train & Test Accuracy**
 - 100 Samples; Train: 0.887 - Test: 0.485
 - 1,000 Samples; Train: 0.837 - Test: 0.644
 - 5,000 Samples; Train: 0.836 - Test: 0.719
 - 10,000 Samples; Train: 0.835 - Test: 0.778



ViT Training Results

100 samples per class

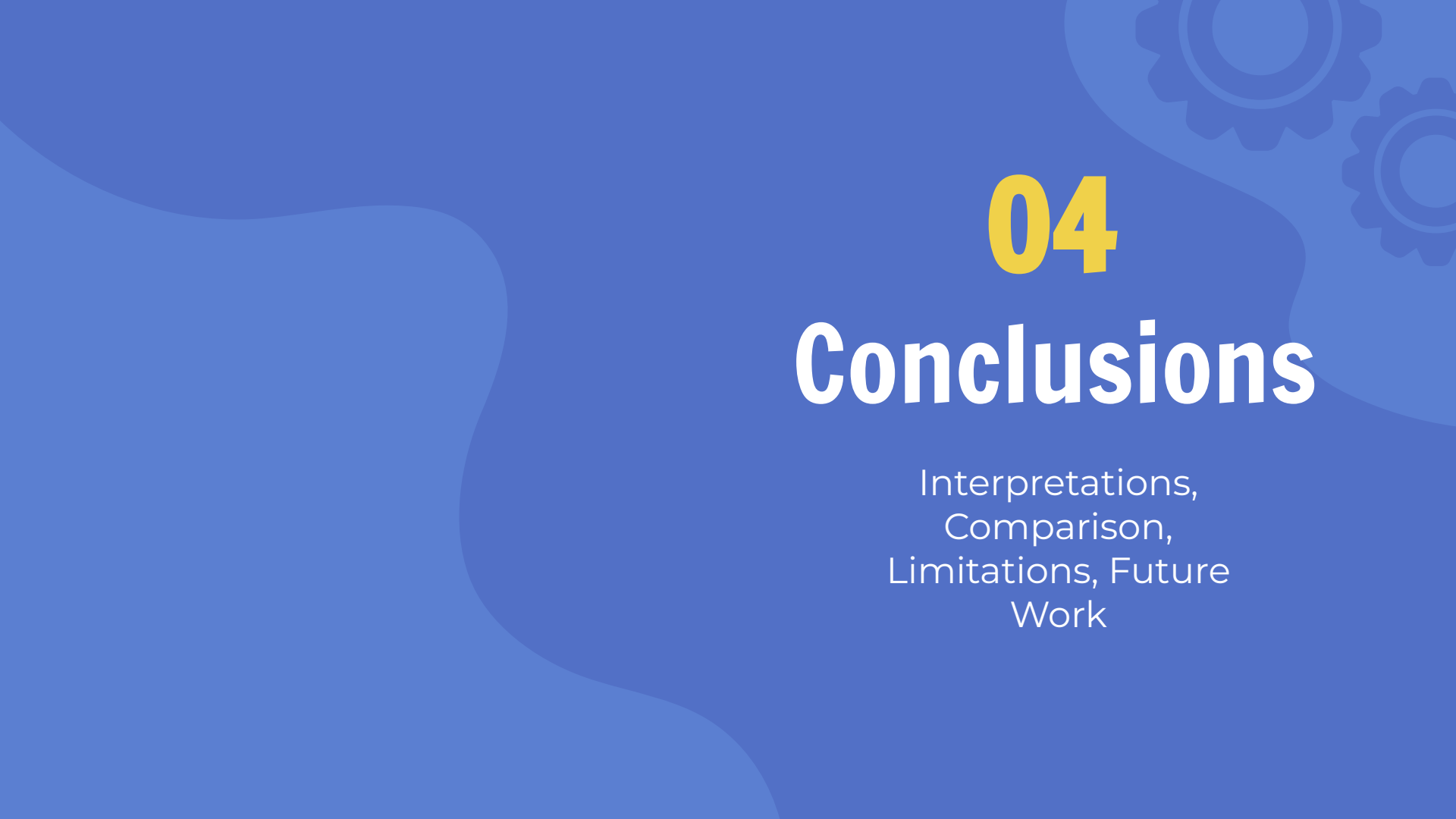
Final classification report:				
	precision	recall	f1-score	support
bandage	0.37	0.65	0.47	20
baseball	0.47	0.35	0.40	20
basketball	0.28	0.45	0.35	20
bat	0.24	0.25	0.24	20
bread	0.50	0.10	0.17	20
camera	0.36	0.65	0.46	20
cat	0.00	0.00	0.00	20
clock	0.38	0.65	0.48	20
compass	0.33	0.05	0.09	20
computer	0.11	0.10	0.10	20
crab	0.53	0.45	0.49	20
dog	0.00	0.00	0.00	20
fan	0.05	0.05	0.05	20
horse	0.00	0.00	0.00	20
laptop	0.00	0.00	0.00	20
lollipop	0.73	0.55	0.63	20
mushroom	0.25	0.35	0.29	20
octagon	0.50	0.75	0.60	20
panda	0.22	0.80	0.35	20
rollerskates	0.42	0.85	0.57	20
shoe	0.50	0.10	0.17	20
soccer ball	0.43	0.30	0.35	20
stop sign	0.42	0.25	0.31	20
strawberry	0.67	0.30	0.41	20
television	0.14	0.30	0.19	20
zebra	0.50	0.35	0.41	20
accuracy			0.33	520
macro avg	0.32	0.33	0.29	520
weighted avg	0.32	0.33	0.29	520

1000 samples per class

Final classification report:				
	precision	recall	f1-score	support
bandage	0.51	0.45	0.48	200
baseball	0.00	0.00	0.00	200
basketball	0.33	0.76	0.46	200
bat	0.48	0.50	0.49	200
bread	0.64	0.19	0.29	200
camera	0.43	0.57	0.49	200
cat	0.39	0.04	0.08	200
clock	0.49	0.41	0.44	200
compass	0.26	0.47	0.34	200
computer	0.35	0.07	0.11	200
crab	0.59	0.39	0.47	200
dog	0.21	0.07	0.11	200
fan	0.43	0.47	0.45	200
horse	0.21	0.66	0.32	200
laptop	0.26	0.27	0.26	200
lollipop	0.70	0.73	0.72	200
mushroom	0.46	0.38	0.42	200
octagon	0.76	0.81	0.78	200
panda	0.42	0.76	0.54	200
rollerskates	0.59	0.79	0.68	200
shoe	0.35	0.59	0.44	200
soccer ball	0.36	0.32	0.34	200
stop sign	0.60	0.39	0.47	200
strawberry	0.74	0.31	0.44	200
television	0.32	0.31	0.31	200
zebra	0.60	0.13	0.21	200
accuracy			0.42	5200
macro avg	0.44	0.42	0.39	5200
weighted avg	0.44	0.42	0.39	5200

5000 samples per class

Final classification report:				
	precision	recall	f1-score	support
bandage	0.54	0.52	0.53	1000
baseball	0.45	0.51	0.48	1000
basketball	0.71	0.13	0.21	1000
bat	0.56	0.45	0.50	1000
bread	0.52	0.39	0.44	1000
camera	0.46	0.74	0.57	1000
cat	0.38	0.30	0.33	1000
clock	0.67	0.32	0.44	1000
compass	0.27	0.60	0.38	1000
computer	0.48	0.19	0.27	1000
crab	0.52	0.63	0.57	1000
dog	0.27	0.12	0.17	1000
fan	0.40	0.63	0.49	1000
horse	0.41	0.43	0.42	1000
laptop	0.46	0.35	0.40	1000
lollipop	0.82	0.66	0.73	1000
mushroom	0.53	0.66	0.59	1000
octagon	0.71	0.88	0.79	1000
panda	0.71	0.59	0.64	1000
rollerskates	0.74	0.84	0.79	1000
shoe	0.65	0.46	0.53	1000
soccer ball	0.38	0.49	0.43	1000
stop sign	0.52	0.63	0.57	1000
strawberry	0.56	0.63	0.60	1000
television	0.54	0.30	0.39	1000
zebra	0.43	0.66	0.52	1000
accuracy			0.50	26000
macro avg	0.53	0.50	0.49	26000
weighted avg	0.53	0.50	0.49	26000

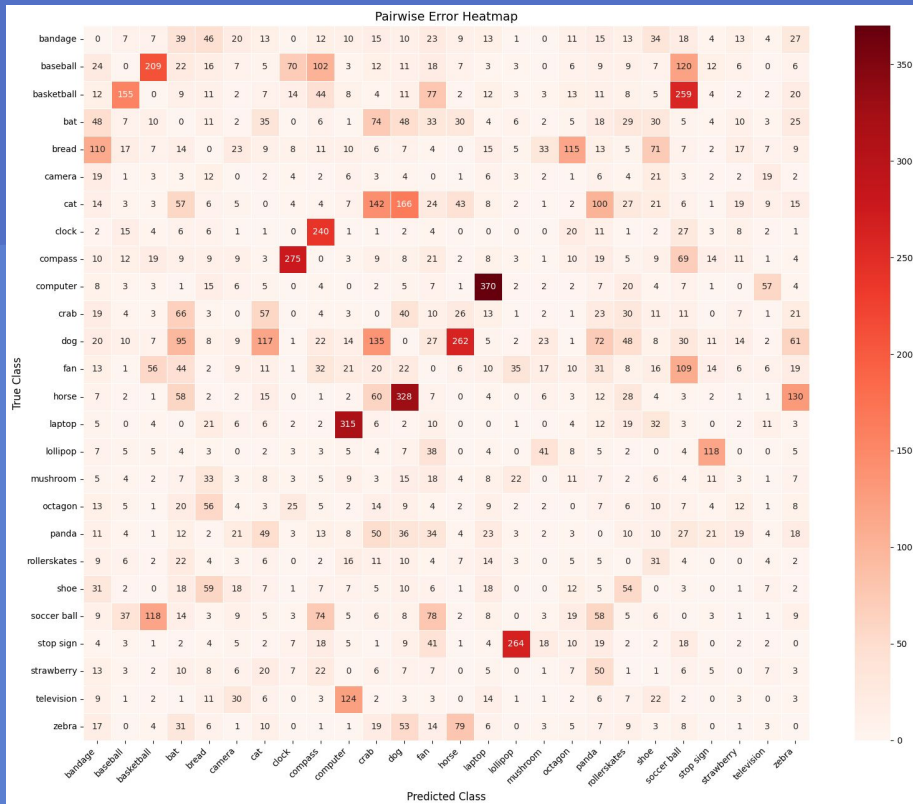


04

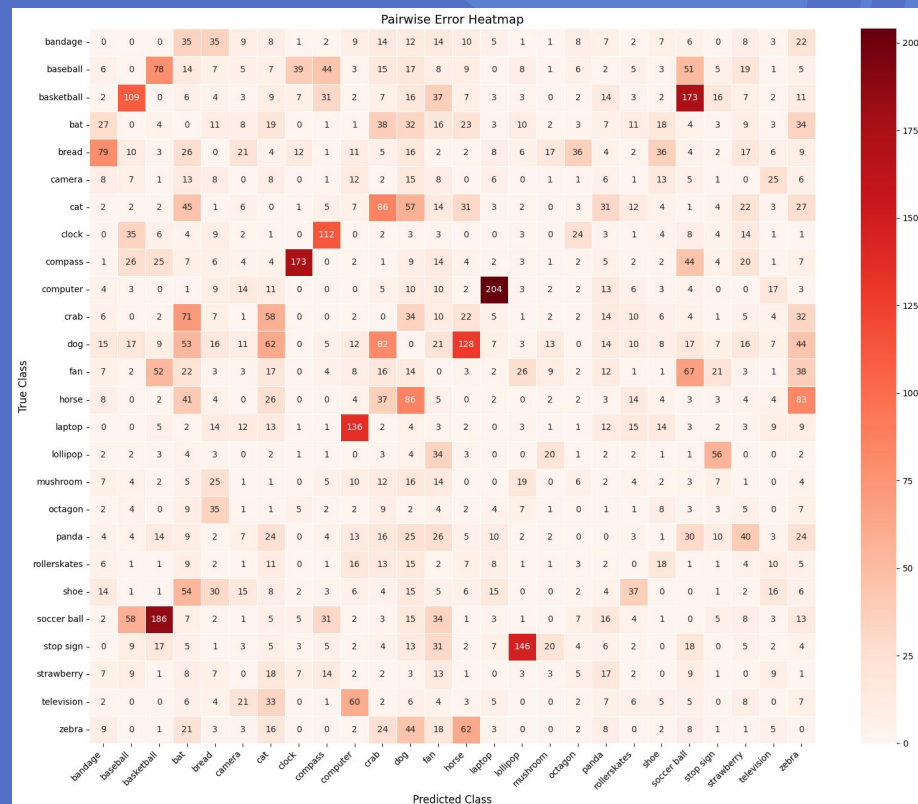
Conclusions

Interpretations,
Comparison,
Limitations, Future
Work

Pairwise Error Heatmaps

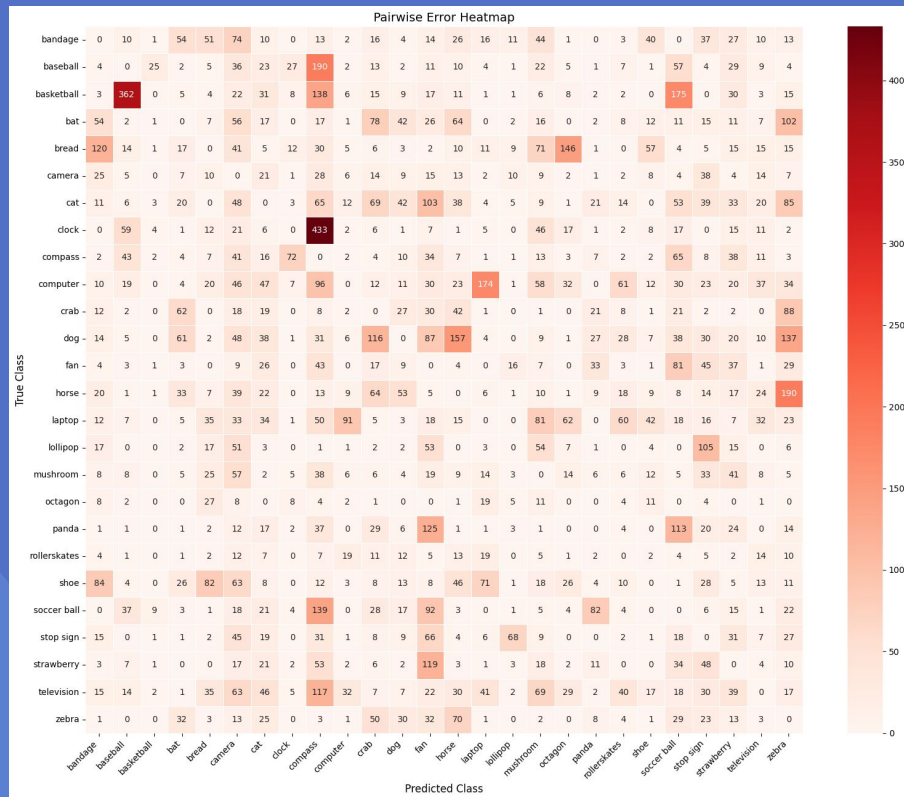


1. Pairwise Error Heatmap for CNN



2. Pairwise Error Heatmap for SVM

Pairwise Error Heatmaps



3. Pairwise Error Heatmap for ViT

Interpretations of Model Results

1. CNN

- Fastest training times between the models
- Predicted similar classes had high confusion

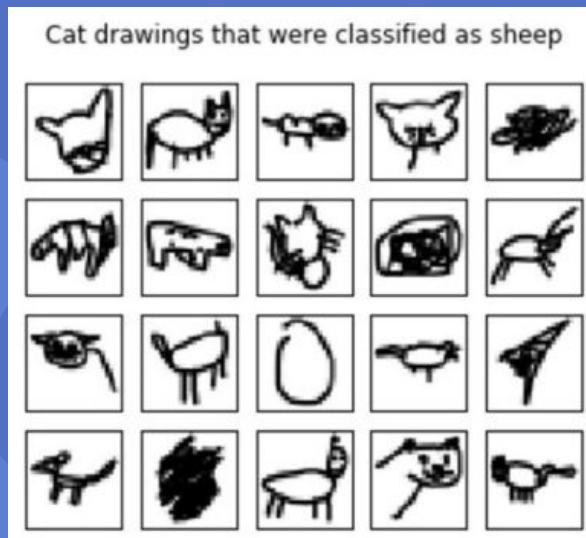
2. SVM

- Overfitting on smaller data samples
- Better generalization on larger data samples
- Poor scalability for larger data samples, drastic increase in training time
- Good accuracy overall but struggles with similar classes

3. ViT

- With smaller data it never learned the differences between similar classes.
- When given strongly distinct classes it preformed well

Limitations



- **Not able to train/evaluate models on the full Google Quick, Draw! Dataset**
 - Very large dataset size (50 million drawings)
 - Resource constraints in Google Colab
- **Performance plateau around 70%-80%**
 - Dataset may be too ambiguous for high overall accuracy and performance
 - Many very inconsistent drawings
 - Many classes inherently visually similar
- **28×28 binary representation of the drawings**
 - Simplified the images to low-resolution to help with computational cost, but also removes many details that could help with distinguishing similar classes

Acknowledgements

Google's Quick, Draw! Dataset:

<https://github.com/googlecreativelab/quickdraw-dataset/blob/master/README.md>

Dosovitskiy, Alexey, et al. "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale." ArXiv:2010.11929 [Cs], 22 Oct. 2020,

arxiv.org/abs/2010.11929,

<https://doi.org/10.48550/arXiv.2010.11929>.

- <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>
- <https://www.geeksforgeeks.org/machine-learning/support-vector-machine-algorithm/>
- <https://www.geeksforgeeks.org/machine-learning/pca-and-svm-pipeline-in-python/>